

 INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE SÃO PAULO CAMPUS – São Paulo		
Disciplina: SPOWEB1		
Desenv. Web 1	Pesquisa técnica- Laços de repetição	Data.: 30/04/2026
Nome do Aluno: Rafaella de Souza Sant" Prontuário: Sp3194434		
Professora: Claudete de Oliveira Link do GitHub: https://github.com/Rafa-Santanna/2026-IFSP/blob/main/Web/Pesquisa_La%C3%A7os_de_repeti%C3%A7%C3%A3o_213_RafaellaDeSouzaSantanna.pdf		

Enunciado: Para cada item abaixo, realize uma pesquisa técnica e entregue:

1. Sintaxe: Como escrever.
 2. Funcionamento: Explicação do comportamento da estrutura.
 3. Exemplo Prático: Código funcional. 4. Desafio + Gabarito: Uma questão criada por você com a solução
- Tópico 1: O Método forEach
 - Tópico 2: O Laço for...
 - Tópico 3: O Laço for...of
 - Tópico 4: O Laço for await...of
 - Tópico 5: O Comando break
 - Tópico 6: O Comando continue

1 INTRODUÇÃO

Essa pesquisa tem como objetivo apresentar a sintaxe, funcionamento, exemplo prático é um desafio de estruturas de repetição e comandos de controle de fluxo, que são eles respectivamente: ForEach(), Laço for...in, laço for...of, laço for await...of e os comandos break e continue.

2 ESTRUTURAS DE REPETIÇÃO

2.1 MÉTODO ForEach()

O método ForEach() executa uma função para cada item presente em um array

2.1.1 Sintaxe

A sua syntax é bem simples.

NomedoArray.forEach(função([ValorAtual, índice, array]), ArgumentoThis)

- A função é aquela que será executada em cada elemento do array e ela pode ter três parâmetros:
 - ValorAtual- É obrigatório. Valor da posição atual.
 - índice- É opcional. Posição do array atual
 - ArgumentoThis- É opcional. Usada no momento de executar a função callback (argumento para outra função, sendo executado após a conclusão da primeira).

2.1.2 Funcionamento

O ForEach no JavaScript é um método de array utilizado para percorrer cada elemento de uma lista e executar uma função de callback para cada item que está dentro do array. Facilitando então a automação de tarefas repetidas sem precisar gerenciar índices manualmente, como a utilização do for tradicional.

2.1.3 Exemplo prático

```
var numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
function tabuadaDe2(item) {  
  console.log(item*2);  
}  
numeros.forEach(tabuadaDe2)
```

Neste primeiro exemplo, podemos observar que a função fará a multiplicação por 2, de cada item, ou seja, o resultado será igual a 0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20.

Podemos observar também a syntax: NomedoArray.forEach(função).

```
var numeros = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10];  
var total = 0;  
function somar(item, indice) {  
  total += item;  
  numeros[indice] = total;  
}  
numeros.forEach(somar);  
console.log(numeros);
```

O valor retornado é: [0, 1, 3, 6, 10, 15, 21, 28, 36, 45, 55]. Pois, nesse exemplo, será somado todos os números presentes no array, ou seja, o total se refere ao índice, então o resultado é a soma de cada número.

2.1.4 Desafio + Gabarito

Desafio) Crie um código, utilizando forEach, que percorra um array e fale quais palavras começam com "R".

Gabarito)

```
var objetos= ["Rastelo", "Castelo", "Roupa", "Paralelepipedo"]
var ObjetosIniciadasComR = [];
var novoIndice = 0;

function selecionarObjetos(item, indice) {
  if (objetos[indice].indexOf("R") == 0) {
    objetosIniciadasComR[novoIndice] = objetos[indice]; novoIndice++;
  }
}

objetos.forEach(selecionarObjetos);
console.log(objetosIniciadasComR);
```

Neste desafio, foi proposto que deixássemos somente as palavras que inicializam com R, por isso utilizamos o forEach, para que ele percorra cada palavras e veja se seu índice 0 é igual a R, se sim ele é adicionado dentro de ObjetosInicializadoscomR[]. Então a resposta final desse desafio seria Rastelo e Roupa.

2.2 LAÇO FOR...IN

O laço for..interage sobre propriedades enumeradas de um objeto, como em chaves.

2.2.1 Sintaxe

Sua sintaxe também é bem simples.

```
for (variavel in obejto){}
```

Sendo que os objetos são o que estão com as propriedades enumeradas.

2.2.2 Funcionamento

Esse laço além de ser utilizado em chaves, onde se tem o nome e sua característica como "Estado: minas gerais", ele também é utilizado para saber em qual posição o objeto se encontra dentro de um array.

2.2.3 Exemplo prático

```
const pessoa = {
  nome: 'Bruno',
  idade: 25}
for (const index in pessoa) {
  console.log(`${index} - ${pessoa[index]}`) }
// nome - Bruno | idade - 25
```

Nesse exemplo é utilizado uma chave, e o for in é usado para mostrar a chave e sua característica, no caso “nome: Bruno”. O index vai ser utilizado para “nome” e a pessoa[index] é o que está escrito no index “Bruno”.

Também poderíamos ter um exemplo na qual a const pessoa =[‘Rafaella’, ‘Claudete’].

O resultado seria:

0 Rafaella

1 Claudete.

2.2.4 Desafio + Gabarito

Desafio) Crie um código que mostre registro de uma pessoa, contendo (Nome, idade, cargo e salário). O resultado deve ser exibido da seguinte forma usando for in (Nome: x, Idade:y...)

Gabarito)

```
const pessoa = {
  nome: "João",
  idade: 30,
  Cargo: "Desenvolvedor"
  Salario: 5.000
};
```

```
for (let chave in pessoa) {
  console.log(chave + ": " + pessoa[chave]);
}
```

Nesse desafio foi proposto que mostrássemos os dados de uma pessoa, por isso utilizamos o for..in, que atua como uma chave e facilita a saída de dados.

No exemplo, a palavra chave são os identificadores como (Nome, idade, cargo e salário) e a pessoa é aquilo que cada identificador carrega.

2.3 LAÇO FOR...OF

O laço `for...of` é uma estrutura de repetição usada para iterar sobre valores de objetos iteráveis. Ele é utilizado por ser mais rápido pois acessa o valor direto de cada elemento e não utiliza índices.

2.3.1 Sintaxe

A sintaxe desse laço é dada por

```
for(TipoDeVariavel Elemento of NomeDoArray){  
}
```

O “Elemento” é uma variável criada para auxiliar o `for`.

2.3.2 Funcionamento

Esse laço funciona quando queremos percorrer um array e mostrar o seu conteúdo, mas o índice não é necessário, ou seja, pelo índice não ser necessário utilizamos esse laço invés do `for`. Com isso o projeto fica mais limpo e ágil.

2.3.3 Exemplo prático

```
const numeros = [1,2,3,4,5];  
for(let numero of numeros) {  
  console.log(numero);  
}
```

Nesse exemplo simples acima, podemos perceber como ele é utilizado. É criado um array chamado `numeros` e queremos que ele mostre os números desse array, para isso utilizamos o `for...of` que vai criar uma variável “Número” que “guardará” cada elemento do array. Então, por fim, é mostrado o número que resultará em (1, 2, 3, 4, 5).

2.3.4 Desafio + Gabarito

Desafio) Crie um código que exiba apenas os números pares e maiores que 10 de um array utilizando `for...of`.

Gabarito)

```
const numeros = [5, 12, 8, 13, 20, 3, 10, 7, 22];
```

```
console.log("Números pares maiores que 10:");
```

```
for (const numero of numeros) {  
  if (numero > 10 && numero % 2 === 0) {  
    console.log(numero);  
  }  
}
```



Neste exemplo criamos um array chamado `números`, a partir dele iremos analisar quais são maiores que 10 e pares. Para isso criamos o `for..of` que cria a variável auxiliar o “numero” e ela guarda cada elemento do array, que vai então, para a verificação do `if` que vê se ele é maior que 10 e par, se for verdadeiro ele mostra esses números.

2.4 LAÇO FOR AWAIT...OF

É uma estrutura de repetição utilizada para iterar objetos iterável assíncronos (como geradores assíncronos) e síncronos. Os assíncronos são valores que demoram para chegar, por isso o “await”, ele espera resolver e depois vai para o próximo.

2.4.1 Sintaxe

A sintaxe segue o padrão do `for..in` e [for..of](#).

```
for await (const variavel of iteravel) {  
  // código  
}
```

2.4.2 Funcionamento

O `for await...of`, ele funciona executando uma coisa de cada vez. Ele é muito utilizado como exemplo em temporizadores onde ele executará um depois executará outro e assim por diante.

2.4.3 Exemplo prático

```
const lista = [Promise.resolve(1), Promise.resolve(2)];
```

```
for await (const item of lista) {  
  console.log(item);  
}
```

Nesse exemplo temos a utilização do `promise`. Além disso, ele esperará o primeiro se resolver e depois fará o segundo e sua saída é (1, 2).

A seguir veremos um exemplo de temporizados como dito anteriormente.

```
function esperar(ms, valor) {
  return new Promise(resolve => {
    setTimeout(() => resolve(valor), ms);
  });
}
```

```
async function rodar() {
  const lista = [
    esperar(1000, "A"),
    esperar(2000, "B"),
    esperar(500, "C")
  ];
```

```
  for await (const item of lista) {
    console.log(item);
  }
}
```

```
rodar();
```

Neste exemplo, podemos observar um temporizador que é executado um de cada vez, ou seja, o outro só será executado quando o anterior já estiver finalizado.

2.4..4 Desafio + Gabarito

Desafio)

Crie um desafio que mostre na tela (A B C) em tempos diferentes, utilizando for [await..of](#).

Gabarito)

```
function promessa(ms, valor) {
  return new Promise(resolve => {
    setTimeout(() => resolve(valor), ms);
  });
}
```

```
async function teste() {
  const lista = [
    promessa(1000, "A"),
    promessa(500, "B"),
    promessa(200, "C")
  ];
```

```
  for await (const item of lista) {
```

```
    console.log(item);  
  }  
}
```

```
teste();
```

Neste código temos a primeira parte que define e faz a função de temporizador, temos a criação do array que será utilizado para escolher o tempo e o que mostrará após isso. É feito também o `for await...of` que mostrará um de cada vez.

3. COMANDO DE CONTROLE DE FLUXO

3.1 COMANDO BREAK

O comando Break ao ser executado em um loop, permite a saída imediata do usuário de dentro do loop.

3.1.1 Sintaxe

Sua sintaxe é simples, pois ele é utilizado dentro do loop no local que ele será executado para saída do loop.

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    break; // Interrompe o laço quando i é igual a 5  
  }  
  console.log(i); // Imprime os números de 0 a 4  
}
```

3.1.2 Funcionamento

Esse comando é muito utilizado quando você precisa sair de um laço quando uma condição específica é executada, além de ser usada para encerrar o código também.

3.1.3 Exemplo prático

```
const numeros = [1, 2, 3, 4, 5];
```

```
for (let i = 0; i < numeros.length; i++) {  
  if (numeros[i] === 3) {  
    break; // para quando encontrar o 3  
  }  
}
```



```
console.log(numeros[i]);  
}
```

No exemplo criamos um array e uma condição, dentro de um loop, onde se o número for igual a 3 ele para e printa o número.

3.1.4 Desafio + Gabarito

Desafio) Crie um código utilizando array, for e break, para que se o número escolhido for ímpar o código para.

Gabarito)

```
const numeros = [2, 4, 6, 7, 8, 10];
```

```
for (let i = 0; i < numeros.length; i++) {  
  if (numeros[i] % 2 !== 0) { // se NÃO for par  
    console.log(numeros[i]);
```

```
    if (numeros[i] > 5) {  
      break;  
    }  
  }  
}
```

Nesse código utilizamos o for para percorrer o array e a condição para verificar se é diferente de par se for break, ou seja, o código finaliza.

3.2 COMANDO CONTINUE

O comando “Continue” é usado dentro de um laço para pular uma iteração atual e continuar com a próxima, ou seja, ele faz o papel contrário do break.

3.2.1 Sintaxe

Sua sintaxe é igual a do break, ou seja, é utilizada dentro de um laço na posição que eu quero que o código pule a iteração atual e passe para a próxima.

```
for (let i = 0; i < 10; i++) {  
  if (i === 5) {  
    continue; // Pula a iteração quando i é igual a 5  
  }  
  console.log(i); // Imprime os números de 0 a 4 e 6 a 9  
}
```

3.2.2 Funcionamento

Geralmente o comando Continue pode ser útil para pular certas partes do código em determinadas condições sem sair do laço.

3.2.3 Exemplo prático

```
const numeros = [1, 2, 3, 4, 5, 6];

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    continue; // pula os pares
  }

  console.log(numeros[i] * 2);
}
```

Neste exemplo, temos um bom funcionamento do continue. Ele é utilizado dentro de uma condição que, se o número for par ele continua e faz número vezes 2, ou seja, ele pegará somente os números ímpares e multiplicará por 2.

3.2.4 Desafio + Gabarito

Desafio)

Crie um código usando “Continue” e “Break ” para que seja verificado se os números de um array são pares e se eles forem confira se são maiores que cinco, se eles forem maiores, pare o código.

Gabarito)

```
const numeros = [2, 4, 6, 7, 8, 10];

for (let i = 0; i < numeros.length; i++) {
  if (numeros[i] % 2 === 0) {
    continue;
  }

  console.log(numeros[i]);

  if (numeros[i] > 5) {
    break;
  }
}
```



Nesse exemplo, é utilizado o `continue` para que, se o número for par ele pule para próxima verificação, que é ver se ele é maior que 5, se ele for o código, irá parar com o `Break`.

4 CONCLUSÃO

Em conclusão, utilizar os laços de repetição é muito importante e evita grandes códigos e repetições. Em Javascript contém também laços de repetições especiais, como visto acima, e eles cumprem funções essenciais para melhor desempenho do código.

5 REFERÊNCIAS BIBLIOGRÁFICAS

DEV MEDIA. *JavaScript forEach: executando uma função para cada elemento de um array*. Disponível em: <https://www.devmedia.com.br/javascript-foreach-executando-uma-funcao-para-cada-elemento-de-um-array/39808>. Acesso em: 21 abr. 2026.

MDN Web Docs. *for...in*. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/for...in>. Acesso em: 22 abr. 2026.

DEV MEDIA. *JavaScript: for, for in e for of*. Disponível em: <https://www.devmedia.com.br/javascript-for-for-in-for-of/41018>. Acesso em: 23 abr. 2026.

MDN Web Docs. *for...of*. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/for...of>. Acesso em: 21 abr. 2026.

MDN Web Docs. *for await...of*. Disponível em: <https://developer.mozilla.org/pt-BR/docs/Web/JavaScript/Reference/Statements/for-await...of>. Acesso em: 22 abr. 2026.

REFACTORANDO. *Break e continue*. Disponível em: <https://plataforma.refatorando.com.br/cursos/curso-basico-javascript/conteudo/3-operadores-e-estruturas-de-controle-de-fluxo/topicos/break-e-continue/>. Acesso em: 23 abr. 2026.

Bons Estudos!!!